

Sokin: Senior QA Analyst (Mobile) Assignment

Candidate: Paulo Goncalves Junior

Email: paulorrg_junior@hotmail.com

Mobile quality-engineering exercise for `automationexercise.com`, treated as the functional reference for a hypothetical native **iOS/Android** e-commerce app.

This repository contains all five deliverables: a test strategy, an exploratory testing report, answers to the product-discovery questions, a working automation framework, and the "how I'd automate this" write-up.

My approach in one minute

1. **Prioritise the buying journey.** For an e-commerce app, value lives in account access, browsing, cart, and checkout. I test and automate that first, and deliberately leave broader testing (many devices, visual checks, other languages, strict performance targets) for later.
 2. **Automate the repeatable, explore the rest.** Stable, repeatable flows are automated, while the device, network, and human dependent parts are checked by hand through exploratory testing.
 3. **Write tests once, run everywhere.** The framework uses Page Objects over a **cross-platform selector map** (`web | android | ios`), so the same specs target the responsive web today and native iOS/Android when the app build exists.
 4. **Prove it runs.** Since there is no real native app (and no emulators are assumed), the framework is genuinely executable today against the responsive site under **mobile-emulated Chrome**. `npm run test:web` runs green with only Node and Chrome. The Appium/iOS/Android targets are fully wired and ready for app artifacts and SDKs.
-

Deliverables map

Deliverable	Where to find it
1. Mobile Test Strategy	The "Mobile Test Strategy" section of this document
2. Exploratory Testing Report	The "Exploratory Testing Report" section of this document
3. Mobile Automation (code and framework)	The project's automation code (the <code>config/</code> and <code>test/</code> folders)
4. How I Would Automate This	The "How I Would Automate This Mobile App" section of this document
5. Product Discovery Questions	The "Product Discovery Questions" section of this document
Evidence (screenshots)	Embedded throughout, within the relevant sections

The automation framework

```
config/
  wdio.shared.conf.ts      # platform-agnostic base (framework, timeouts, isolation hook
  wdio.web.conf.ts         # RUNNABLE TODAY: mobile-emulated Chrome (Node + Chrome only)
  wdio.android.conf.ts     # Appium + UiAutomator2 (needs Android SDK + .apk)
  wdio.ios.conf.ts         # Appium + XCUITest (needs Xcode + .app/.ipa)
test/
  specs/                   # business flows + assertions only (no selectors)
    auth.e2e.ts            #   register a new user end-to-end; reject invalid login
    cart.e2e.ts            #   add multiple products; cart persists across refresh
  pageobjects/             # Page Object Model: actions/intent per screen
  selectors/index.ts       # ONE cross-platform locator map { web, android, ios }
  data/users.ts            # unique-per-run test-data factory
  support/platform.ts      # resolves the active platform
.github/workflows/ci.yml   # PR gate (web-emulation) + scaffolded native jobs
```

Design highlights (full reasoning in the "How I Would Automate This Mobile App" section of this document):

- **Page Object Model:** specs read as intent (`AuthPage.login(...)`), and selectors never leak into tests.
- **Cross-platform selectors:** each element is `{ web, android, ios }`, and native defaults to accessibility ids (stable and accessibility friendly). Only this map changes when the real app lands.
- **Resilient interactions:** a shared `click` waits, centers, and falls back to a DOM click when overlays intercept taps.
- **Test data and isolation:** a unique user per run, and cookies cleared before each web test so order never matters.

Run it

Prerequisites: Node 20+ and Google Chrome. (WebdriverIO v9 auto-manages the matching driver.)

```
npm install
npm run typecheck      # TypeScript: zero errors
npm run test:web       # runnable proof: mobile-emulated Chrome vs. the live site
npm run test:web:headed # same, with a visible browser
```

Expected result (verified on this machine):

```
auth.e2e.ts    ✓ registers a brand-new user end-to-end and signs them in
               ✓ rejects invalid credentials with a clear, user-facing error
cart.e2e.ts    ✓ keeps all added items after a page refresh
Spec Files:    2 passed, 2 total
```

Native targets (framework ready; require the app build + SDKs):

```
ANDROID_APP=/abs/path/app.apk npm run test:android    # Appium 2 + uiautomator2 + emulator
IOS_APP=/abs/path/App.app      npm run test:ios        # Appium 2 + xcuitest + simulator
```

Notes & assumptions

- No real native app exists for this exercise, so "mobile automation" is implemented as a native-shaped framework with a runnable web-emulation proof. The same specs and Page Objects drive all three targets.
- The registration test creates real accounts on the public practice site, using a **unique random email per run** to stay idempotent.
- Environment here: Node v20.11, macOS; no Appium/emulators installed, which is why the web target is the runnable proof.